

课程项目技术报告：横版格斗游戏中的关键软件技术

姓名：李子嘉

学号：2024012325

邮箱：zj-li24@mails.tsinghua.edu.cn

电话：(+86) 13585915409

2026 年 6 月 21 日

目录

摘要	4
1 游戏概览	5
1.1 游戏介绍	5
1.2 开局设置	5
1.3 武器系统	6
1.4 法术系统	6
1.4.1 定身术 (FREEZE)	6
1.4.2 隐身术 (STEALTH)	6
1.4.3 铜头铁臂 (IRON_BODY)	7
1.5 伤害计算公式	7
1.6 词条系统	7
1.7 掉落物与拾取	8
1.8 AI 对手	9
1.9 对局结算	9
1.10 内容规模盘点	9
2 技术一：面向对象技术在游戏系统中的应用	10
2.1 游戏中引出的问题	10
2.2 技术方案与设计思路	10
2.2.1 Entity 基类：定义最小接口	10
2.2.2 三个典型子类，三种不同的行为实现	11
2.2.3 多态在游戏循环中的实际使用	12
2.2.4 Player 类的组合模式	12
2.2.5 什么时候不用继承——武器系统的务实选择	13

2.3	代码验证	14
2.4	替代方案分析	14
2.5	小结	15
3	技术二：事件驱动模块间松耦合通信	16
3.1	游戏中引出的问题	16
3.2	技术方案与设计思路	16
3.2.1	信号槽在本项目中的角色	16
3.2.2	动态连接与断开	18
3.2.3	并非所有通信都用信号	18
3.3	代码验证	18
3.4	替代方案分析	19
3.5	小结	20
4	技术三：数据驱动的游戏玩法可扩展性	21
4.1	游戏中引出的问题	21
4.2	技术方案与设计思路	21
4.2.1	三个层次的数据驱动设计	21
4.2.2	SpellState：法术状态的纯数据结构	24
4.3	代码验证	25
4.4	替代方案分析	26
4.5	小结	26
5	技术四：有限状态机驱动的 NPC 决策	27
5.1	游戏中引出的问题	27
5.2	技术方案与设计思路	27
5.2.1	三层决策架构	27
5.2.2	状态转换的决策逻辑	28
5.2.3	特殊分支的实际价值	29
5.3	代码验证	29
5.4	替代方案分析	30
5.5	小结	31
6	技术五：实时游戏主循环的时序设计	32
6.1	游戏中引出的问题	32
6.2	技术方案与设计思路	32
6.2.1	固定的帧更新流水线	32
6.2.2	帧时间补偿（dt 传递）	33
6.2.3	对象生命周期管理	34
6.3	代码验证	34
6.4	替代方案分析	35

目录	3
6.5 小结	35
7 技术六：跨平台构建系统的抽象与配置	36
7.1 游戏中引出的问题	36
7.2 技术方案与设计思路	36
7.2.1 CMake 如何解决本项目的问题	37
7.3 代码验证	38
7.4 替代方案分析	39
7.5 小结	39
A 编译与运行环境	39
A.1 开发与运行环境	39
A.2 项目文件清单	40
A.3 编译步骤	40
A.4 编译验证	41
A.5 运行与操作说明	41
结语	42

摘要

本报告围绕题目 3.19（横版格斗对战游戏）的工程实践，提炼出六项在项目中起到关键作用的软件技术。这些技术并非预先选定的“展示项目”，而是从游戏的实际需求出发推导出的必需品——一个模块复杂、交互密集的格斗游戏天然要求这些技术作为支撑。每项技术按照“游戏需求引出的问题→为什么简单方案不够→技术方案与设计思路→代码验证→替代方案分析”的结构展开。六项技术分别为：面向对象技术在游戏系统中的应用、事件驱动的模式间松耦合通信、数据驱动的玩法可扩展性、有限状态机驱动的NPC决策、实时游戏主循环的时序设计、以及跨平台构建系统的抽象与配置。

1 游戏概览

在进入技术分析之前，本节完整介绍游戏的玩法、模式和系统，为后续各技术模块提供背景信息。

1.1 游戏介绍

本项目是一个基于 Qt 图形界面的横版格斗对战游戏。两名角色在 2D 平台上进行对战——通过键盘操控角色移动、跳跃、近战/远程攻击、切换武器和释放法术，目标是将对手生命值削减至零。

游戏支持三种对局模式：

- **人机对战 (AI)**：玩家操控 P1，P2 由 AI 控制。适合单人快速体验。
- **双人对战 (PVP)**：两名玩家使用同一键盘的不同键位分别操控 P1 和 P2。P1 键位为 WASD（移动）+ F（攻击）+ G/C（切换武器/法术），P2 键位为 IJKL（移动）+ O（攻击）+ N/U（切换武器/法术）。由于两人共用同一键盘，程序在设置界面检测按键冲突并在对局中过滤矛盾输入。
- **无尽模式 (ENDLESS)**：玩家操控 P1 连续对抗 AI 对手，每轮结束后从三个随机词条中选择一个增益，AI 伤害逐轮递增。玩家阵亡即游戏终止，显示结算面板（总伤害、击杀数、生存时间）。无尽模式中 AI 拥有 `endlessImmortal` 属性——阵亡后有 1 秒保护期才判定真正死亡。

1.2 开局设置

启动程序后进入主菜单，点击“开始”进入设置页面。设置页面提供以下选项：

- **游戏模式**：人机 / 双人 / 无尽，三种模式互斥选择。
- **总局数 (BO)**：支持 Best of 3 / 5 / 7 / 自定义。BO3 意为率先获得 2 胜者赢得整场对局，每小局结束返回设置页复用原配置。
- **场景选择**：默认场景（常规平台布局）或草地场景（草丛可隐藏角色——不移动时自动蹲下进入隐匿状态）。
- **数值参数**：血量上限（默认 200）、实心球初始数量（默认 3）、步枪弹药（默认 5）、狙击弹药（默认 2）。可通过预设档位或手动输入调整。
- **法术选择**：P1 和 P2 各选一个法术——定身、隐身（仅人机可用）、铜头铁臂，或选“无法术”。双人模式中隐身不可选。选完后系统根据法术类型自动筛选对应该法术的词条池。
- **AI 法术策略**（仅人机/无尽）：CONFIG_PICK（使用 P2 指定法术）、COUNTER_PICK（根据玩家法术加权对策抽样）、RANDOM_PICK（随机）。

点击“开始对局”后进入游戏主界面，60FPS 的帧循环即刻启动。画面上方显示双方状态栏——血条、武器图标、词条摘要、法术冷却进度——左上角为 P1，右上角为 P2。

1.3 武器系统

每位角色初始拥有拳击（无弹药限制），可通过拾取地面掉落物获取新武器。所有武器存放在**武器槽**中（默认 1 槽，可通过“百宝囊”词条扩展），按 C/N 键循环切换。五种武器的基础数据如下：

武器	类型	伤害倍率	弹药限制	弹道
拳 击 (punch)	近战	5	无限	角色前方 40px 碰撞框
小刀 (knife)	近战	15	无限	角色前方 40px 碰撞框
实 心 球 (ball)	远程	1	初始 3 枚	抛物线，受重力
步枪 (rifle)	远程	30	初始 5 发	水平直线
狙 击 枪 (sniper)	远程	60	初始 2 发	水平直线

表 1: 五种武器的参数对比。远程倍率指枪械子弹的伤害系数（狙击枪单发 60，步枪单发 30），实心球伤害则由动能公式另行计算（见下文伤害公式）。

武器可以通过拾取掉落物补充弹药或替换为更高优先级的武器（AI 会自行评估武器强度决定是否拾取，若武器槽有空位则不会替换而是新增）。

1.4 法术系统

每名角色在开局时选定一个法术（或“无法术”），对局中按 Q（P1）/ U（P2）键释放。每种法术有独立的冷却时间，冷却期间按键无效，进度在状态栏以圆环形式显示。游戏包含三种法术：

1.4.1 定身术（FREEZE）

- **释放条件：**自身不在受击硬直中。
- **效果：**对手 5 秒内无法移动和攻击，速度清零，不受重力。**定身与隐身不会同时存在**——若对手正在隐身则解除其隐身。定身期间可破除铜头铁臂。
- **解除条件：**对定身目标造成一定量伤害后定身破碎，目标被击退并进入硬直。破碎时施法者若有“破而后立”词条则冷却时间减少 5 秒。
- **冷却时间：**40 秒（从释放时计算）。
- **定身增伤：**被定身的目标受到攻击时触发额外伤害乘区。

1.4.2 隐身术（STEALTH）

- **释放条件：**自身不在受击硬直中，且未被定身。仅限人机/无尽模式（双人模式不可选）。

- **效果：**角色变半透明，AI 失去对其视野（AI 将前往玩家最后已知位置巡逻）。持续 15 秒。期间可正常移动、攻击、拾取物品。
- **解除条件：**被定身时强制解除隐身，但按剩余时间比例返还冷却时间。若拥有“破影一击”词条，隐身时长减半，首次攻击伤害随隐身已过时间线性增长（ $1.0\times$ 至 $2.0\times$ ），攻击后立即解除隐身。
- **冷却时间：**45 秒（从隐身开始计算）。

1.4.3 铜头铁臂（IRON_BODY）

- **效果：**激活后，近战攻击命中自身时——反弹此次伤害，攻击者被硬直 1 秒。如果攻击是远程（实心球/子弹），则将其以一定速度反弹回去且 `parentId` 变为防守方。
- **弱点：**被定身时铜头铁臂直接解除。
- **词条加成：**“荆棘”在反弹时对攻击者额外造成其攻击伤害的 50%；“越战越勇”在反弹成功后减 CD 5 秒；“硬化”激活期间移速 +30%。

1.5 伤害计算公式

攻击命中后的伤害结算遵循以下流水线（各乘区独立）：

1. **基础攻击力 *BaseAtk*：**读取攻击者的基础攻击力，叠加通用伤害加成（`damageBonusMultiplier`）、两败俱伤（ $1.5\times$ ）、破影一击（ $1.0\times$ 至 $2.0\times$ 线性增长）等乘区修正。
2. **武器倍率：**乘以武器对应倍率——近战拳击 $\times 5$ ，小刀 $\times 15$ ；远程步枪子弹 $\times 30$ ，狙击子弹 $\times 60$ ；实心球 $\times 1$ 。
3. **武器专精：**在武器倍率后继续叠加对应武器的专精加成（如拳击 +80%、小刀 +40% 等词条）。
4. **实心球动能计算（仅实心球）：**不直接使用武器倍率，而根据发射初速和球的动能公式通过 sigmoid 映射得到伤害值，且单次伤害上限不超过目标最大生命值的 50%。
5. **定身受击增伤：**若防守方处于定身状态，触发 `freezeDmgMultiplier` 额外乘区。
6. **护甲与受击减伤：**最后再进入护甲减伤 / 受击减伤阶段，统一写成 *damage* 的后置衰减项。

核心规则：所有词条均按独立乘区结算，词条等级叠加视为相同乘区内的线性累加（如两个“伤害+25%”词条使 `damageBonusMultiplier` 从 1.0 变为 1.5，而非 1.25×1.25 ）。

1.6 词条系统

词条（Modifier）是角色的被动增益，通过拾取地面上的词条卡片获得。人机/双人模式中，词条仅通过掉落物获取；无尽模式中，每轮结束自动弹出三选一词条面板。

词条分为两大类：**通用词条**（13 种，所有法术均可获取）和**法术专属词条**（每种法术 3-4 种，仅持有该法术时进入词条池）。完整列表如下：

词条名称	类型	效果	备注
急转轮回	通用	法 术 冷 却 5秒/级	- 可叠加
厚积薄发	通用	自 身 血 量 上 限 +20%/级	基于初始血量
釜底抽薪	通用	对 手 血 量 上 限 20%/级	- 施加到对手
风驰电掣	通用	自 身 移 速 +20%/级	—
缚风捉影	通用	对 手 移 速 15%/级	- 施加到对手
铁拳无敌	通用	拳 击 伤 害 +80%/级	可叠加
刀光剑影	通用	小 刀 伤 害 +40%/级	可叠加
石破天惊	通用	实 心 球 伤 害 +20%/级	可叠加
弹无虚发	通用	枪 械 伤 害 +30%/级	步枪/狙击共用
锋芒毕露	通用	通用伤害加成 +25%/级	武器倍率前乘区
金刚不坏	通用	伤 害 减 免 +20%/级	上限 60%
两败俱伤	通用	造成与受到伤 害各 +50%	唯一
百宝囊	通用	额外获得 1 武 器槽位	可叠加

表 2: 13 种通用词条。标注“唯一”的词条仅可获得一次（不可叠加等级）。未标注的为可叠加词条。

法术专属词条按法术分类——定身法有“长缚”“缚中伤”“破而后立”等；隐身法有“长隐”“影疾”“潜息”“破影一击”等；铜头铁臂有“铜骨”“荆棘”“越战越勇”“硬化”等（完整列表见源码 `modifierdata.h`）。

1.7 掉落物与拾取

游戏进行中，地面会定时刷新或在击杀敌人后掉落物品：

- **武器：**小刀、实心球、步枪、狙击枪——走近自动拾取，替换或补充对应槽位。
- **词条卡片：**走近自动拾取，调用 `applyModifier()` 立即生效。

- **血瓶/药包**：立即回复一定生命值、获得生命恢复效果或者临时增益。

所有掉落物有 300 帧（约 5 秒）存活时间，前 180 帧正常显示，最后 120 帧闪烁提示即将消失。

1.8 AI 对手

人机模式和无尽模式中的 P2 由 GameAI 控制。AI 具有两种随机人格（出生时随机分配）——“背水一战”（低血量时仍倾向进攻）和“游走机动”（优先搜寻物资）——使得每局 AI 行为不可预测。AI 能自主判断何时追击、何时找武器、何时施法、何时撤退，具体决策机制见技术四。

1.9 对局结算

单局模式中一方阵亡即结束，显示胜负画面，玩家按任意键返回主菜单。BO 模式下多局累计比分，先达到目标胜场数者赢得整场。无尽模式中玩家阵亡后显示结算面板——包含总伤害输出、总击杀数、生存时间和历史最佳记录——可选择“再来一局”复用配置或“返回菜单”。

1.10 内容规模盘点

在进入技术分析前，盘点这个“故意设计得复杂”的游戏到底有多少内容，以便理解六项技术在项目中的必要性：

- 3 种游戏模式 × 2 种场景 = 6 种对局配置
- 5 种武器 × 各自的弹药、倍率、弹道行为
- 3 种法术（定身/隐身/铜头铁臂）× 各异的激活/冷却/解除机制
- 25+ 种词条（13 通用 + 每种法术 3-4 种专属）× 独立乘区叠加规则
- 8 个 AI 状态 + 2 种人格 + 特殊分支
- 11 个软件模块，40+ 源文件，跨平台构建

如果这些内容用手写 if-else 和全局变量来管理，代码早已不可维护。接下来的六章将逐一展示本项目用什么技术方案来应对这些复杂度。

2 技术一：面向对象技术在游戏系统中的应用

2.1 游戏中引出的问题

一个横版格斗游戏存在多种可交互对象：玩家角色、AI对手、实心球（投掷物）、步枪子弹、狙击枪子弹、掉落物（武器/血瓶/词条）。尽管这些对象的行为差异巨大——子弹直线飞行、实心球受重力走抛物线、掉落物静止不动、玩家角色有复杂动画——它们共享一组基本需求：

1. 每帧都要更新位置（update）
2. 每帧都要绘制到屏幕（draw）
3. 需要提供碰撞箱供战斗系统判定（hitbox）
4. 需要知道自己何时该被销毁（shouldBeRemoved）
5. 与玩家发生碰撞时触发特定行为（onCollideWithPlayer）
6. 需要暴露伤害值和武器类型供伤害计算（getDamage、getWeaponType）

假设不用面向对象技术，最直接的替代方案是为每种对象类型写一套独立的处理逻辑——每个游戏帧里，函数 A 处理球的运动，函数 B 处理子弹的运动，函数 C 处理掉落物的计时，函数 D 挨个绘制所有东西。这意味着：每增加一种新实体类型，就要在游戏循环的每个环节追加新的分支。本项目目前只有 4 种实体（球、步枪子弹、狙击子弹、掉落物），但如果按上述“每类一套逻辑”的做法，游戏循环中已经需要写 4 套相似却不完全相同的处理路径；扩展到 10 种实体时，维护成本将按实体类型数 $\times$ 处理环节数增长。这正是本项目采用继承体系要避免的局面。

2.2 技术方案与设计思路

本项目的核心思路是：用一个抽象基类 `Entity` 定义“所有可交互对象”的统一接口，每个具体实体类型实现自己的行为。关键设计决策如下。

2.2.1 `Entity` 基类：定义最小接口

`entity.h` 中定义的基类只包含所有实体共有的数据和纯虚函数接口（参见文件 `entity.h`）：

- 共有的数据：位置（`position`）、速度（`velocity`）、重力常量、阻力常量、帧时间 `dt`、所属者 ID（`parentId`）、层级（`layer`）等。这些是任何运动物体都需要的物理属性。
- 纯虚函数接口：`update()`、`draw()`、`hitbox()`、`getDamage()`、`getWeaponType()`、`onCollideWithPlayer()`、`shouldBeRemoved()`。每个具体子类必须实现这些函数。
- 已实现的通用方法：`applyGravity()`（施加重力加速度）、`stop()`（施加阻力减速）、`getPos()`（返回当前位置）、`getDir()`（根据速度方向返回朝向）。这些方法的行为对所有实体一致，放在基类中避免代码重复。

```

class Entity
{
public:
    Entity(QPointF pos, int id=0, float frozenBonus = 1.0f);
    virtual void update() = 0;
    virtual void draw(QPainter& painter) = 0;
    virtual void onCollideWithPlayer(){};
    virtual float getDamage(int id){return 0;};
    virtual QRect hitbox(){};
    virtual Player::WeaponType getWeaponType(){ return Player::WeaponType::punch; };
    virtual bool shouldBeRemoved(){};
    void applyGravity();
    void stop();
    // ...
protected:
    QPointF position;
    QPointF velocity;
    float dt;
    int parentId;
    int layer;
};

```

2.2.2 三个典型子类，三种不同的行为实现

Ball（实心球）继承 Entity，在虚函数中实现了：

- **update()**：根据速度和重力更新位置，递减生命周期计时器，到期后标记删除。
- **getDamage()**：基于动能公式（初速度和质量）计算撞击伤害，并受 frozenBonus 的定身增伤加成。
- **onCollideWithPlayer()**：碰撞后直接标记为待删除（球击中目标即消失）。
- **getWeaponType()**：返回 ball。

Bullet（子弹）继承 Entity，行为与 Ball 明显不同：

- **update()**：仅有水平匀速运动（无重力），靠设定初速度 v_x 直线飞行。
- **getDamage()**：伤害基于发射者的基础攻击力乘以武器倍率，而非动能公式。
- **getWeaponType()**：根据内部 type 字段（1 = rifle, 2 = sniper）返回不同武器类型。
- 步枪子弹和狙击子弹共用同一个 Bullet 类，通过构造函数参数 type 区分行为。

DropItem（掉落物）继承 Entity，行为最简单：

- `update()`: 仅递减存活计时器, 不移动 (位置固定)。
- `shouldBeRemoved()`: 在超时或被玩家拾取后返回 `true`。
- `getDamage()`: 返回 0 (掉落物不会造成伤害)。
- 额外提供 `isCollectedBy(Player*)` 方法判断是否被玩家拾取。

2.2.3 多态在游戏循环中的实际使用

在多处关键路径中, 代码通过 `Entity*` 基类指针操作具体对象, 完全不需要知道运行时类型:

场景一: 统一渲染 (`widget.cpp`)——所有实体通过同一段代码绘制:

```
for (const auto& entity : entities) {
    if (entity) {
        entity->draw(painter);    // 虚函数调用, 每个子类画自己
    }
}
```

场景二: 统一清理过期对象 (`widget.cpp`)——用 `remove_if` 配合虚函数判断:

```
entities.erase(std::remove_if(entities.begin(), entities.end(),
    [](const std::shared_ptr<Entity>& e) {
        return e->shouldBeRemoved();    // 每个子类自己决定何时过期
    }), entities.end());
```

场景三: 战斗碰撞结算 (`combatmanager.cpp`)——战斗管理器只知道自己面对的是 `Entity*`, 通过虚函数获取决策所需信息:

```
void CombatManager::checkEntityVsPlayerCollision(Entity *e, PlayerController *p) {
    const auto weaponType = e->getWeaponType();    // 虚函数
    float rawDmg = e->getDamage(p->getId());        // 虚函数, 球和子弹计算方式完全不同
    float defMult = p->getDefenseMultiplier(e->getWeaponType());
    // ... 统一伤害结算逻辑 ...
}
```

这意味着: `CombatManager` 不需要知道冲过来的是球还是子弹。它只需调用虚函数接口。未来的新武器类型只需新建一个 `Entity` 子类, 战斗管理器不用改一行代码。

2.2.4 Player 类的组合模式

与 `Entity` 继承树并行, `Player` 类采用了另一种 OOP 策略——组合。Player 不继承 `Entity` (玩家有复杂的状态机、动画系统和控制逻辑, 与简单的 `Entity` 接口差异太大), 而是将行为委派给多个成员对象:

- `ModifierState modifiers`: 存储当前激活的所有词条效果 (移速倍率、伤害倍率、减伤率等 30+ 字段), 封装在独立结构体中。

- `SpellState spellState`: 管理法术的激活状态、冷却时间和持续时间。
- `Animation animation`: 管理精灵动画的帧切换、加载和播放，与游戏逻辑完全隔离。
- `State state`: 仅包含 `moveState`（当前动作状态）和 `shootState`（是否在射击）两个字段的结构体。

这是“组合优于继承”的实践：如果把这些都塞进 `Player` 的成员变量，`Player` 会变成 200+ 字段的庞然大物；如果强行用多继承（`Player` 继承 `Modifier`、`Spell`、`Animated`），会把不相关的职责耦合到同一个对象里，而且 C++ 的多继承会引入菱形继承问题和更复杂的内存布局。

2.2.5 什么时候不用继承——武器系统的务实选择

项目中没有为不同武器类型创建子类（如 `class Punch : public Weapon`）。武器类型使用 `enum class WeaponType { punch, knife, ball, rifle, sniper }` 表示，战斗计算中通过 `switch` 分支处理不同武器的倍率和行为。

这个选择是有意为之：

- 武器类型只有 5 种，且行为差异局限在伤害倍率和射程，不需要完整的子类体系。
- 如果为武器建继承树，需要额外设计工厂模式、武器槽管理、切换动画等配套设施，增加大量代码但收益有限。
- 与 `Entity` 不同——`Entity` 的每个子类行为差异足够大（飞行、碰撞、生命周期完全不同），继承的价值明确；而武器的差异只在一两个参数上，用一个 `enum` 加 `switch` 的代码更清晰且更少。

这就是 OOP 的务实应用：当继承能消除重复的控制流时使用继承，当数据足以表达差异时使用数据而非子类。

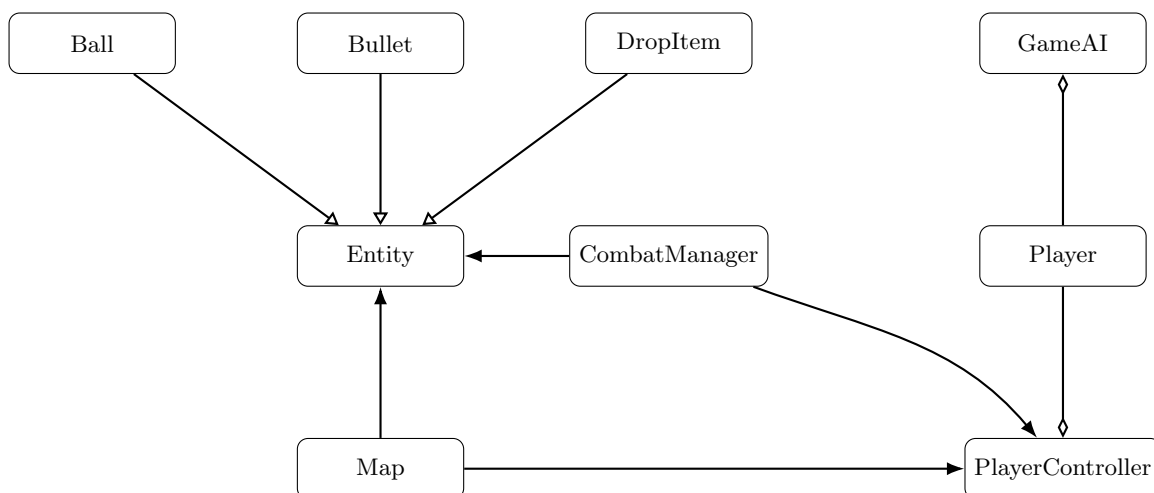


图 1: 核心类关系图。Entity 为基类，Ball、Bullet、DropItem 继承之；Player 与 Entity 分离，通过组合管理复杂状态；CombatManager、GameAI、Map 分别负责战斗结算、AI 决策和寻路，通过依赖关系与其他类协作。

2.3 代码验证

以下验证案例展示了多态和组合在实际运行中的表现：

验证项	输入/触发	预期行为	结论
多态渲染	玩家投掷实心球 + 开枪同时，场景中同时存在 Ball、Rifle Bullet、Sniper Bullet、DropItem	所有实体通过同一段 <code>entity->draw()</code> 正确绘制各自贴图，无需类型判断	通过
多态碰撞结算	Ball 和 Bullet 分别击中玩家	CombatManager 通过 <code>e->getWeaponType()</code> 和 <code>e->getDamage()</code> 获取不同的武器类型和伤害值，伤害公式各不同	通过
多态生命周期管理	等待实体过期	Ball 在飞行 3 秒后消失，Bullet 在飞行一定距离后消失，DropItem 在 300 帧后消失，全部通过同一个 <code>e->shouldBeRemoved()</code> 接口清理	通过
组合模式词条叠加	玩家依次拾取“伤害+20%”和“移速+15%”词条	ModifierState 中各字段独立累加，互不干扰，战斗计算统一遍历生效	通过
继承与组合的边界	新增武器类型“狙击枪”	只需在 WeaponType enum 中加一项 + switch 中加一个倍率分支，不需要新建类或修改 Entity 继承树	通过

2.4 替代方案分析

方案	优势	代价/为何未采用
全部用独立类（无继承）	每个实体类型完全独立，无虚函数开销，编译依赖最小	需要在游戏循环的每个处理环节为每种实体写分支；新增实体需要在 10+ 个位置追加代码。对 4 种以上实体类型基本不可维护

Entity + Player 统一继承树	Player 和 Entity 共享同一套更新/渲染接口	Player 有动画状态机、输入系统、武器槽、词条等大量独有需求，强行套用 Entity 接口会让基类变得臃肿（要么塞进所有可能的方法让大部分子类留空，要么做大量 dynamic_cast）。当前设计中 Player 和 Entity 分离是合理的
ECS 架构（Entity-Component-System）	完全的组件化设计，数据与行为彻底分离；System 遍历 Component 进行统一处理；极致灵活，运行时任意组合	架构复杂度和代码量远超当前需求（需要 Component 管理器、System 调度器、事件总线）；学习和调试成本高；在 6 种实体类型的规模下性价比低
ECS + 当前继承体系混合	更灵活	引入两种范式会增加概念负担和边界不一致，课程周期内不划算

2.5 小结

本项目的 OOP 应用遵循一条务实原则：当多种对象共享生命周期和交互模式时，用继承消除重复；当对象的内部组成复杂但对外接口简单时，用组合管理复杂度；当类型间差异仅在于参数值时，用数据而非子类。Entity 继承树处理了“多种飞行物/掉落物的统一更新渲染清理”，Player 的组合结构处理了“一个复杂角色的词条法术动画状态共存”，武器 enum 避免了不必要的继承开销。这三者结合，构成了项目中 OOP 技术应用的核心。

3 技术二：事件驱动模块间松耦合通信

3.1 游戏中引出的问题

回顾游戏概览中列举的 10 个模块。如果把这些模块之间的信息流画出来，会得到一张相当复杂的图——

- **PlayerController** → **Widget**: 玩家按攻击键 → 创建投掷物实体 (Ball 或 Bullet)，添加到场景中
- **Widget** → **ModifierOverlay**: 无尽模式每轮结束 → 弹出三选一词条面板；玩家做出选择 → 应用词条到玩家角色
- **Widget** → **EndlessResultWidget**: 无尽模式死亡 → 显示结算面板；玩家点击按钮 → 返回菜单或重新开始
- **SetupWidget** → **MainWindow**: 设置完成 → 携带 Session 配置启动游戏页面
- **Widget** → **MainWindow**: 对局结束 → 通知主窗口切换页面
- **PlayerController** → **Widget**: 定身被打破 → 通知 Widget 检查施法者是否有 CDR 词条以减少冷却
- **QTimer** → **Widget::gameLoop**: 每 16ms 触发一次游戏帧，驱动整个游戏循环

如果不加设计地处理这些信息流，每个模块都会直接持有其他模块的指针并调用其方法。以“玩家开枪”为例：**PlayerController** 需要知道 **Widget** 的存在，调用 `widget->spawnBullet(x, y, vx, vy)`；如果后来又增加了音效模块和统计模块需要监听“开枪”事件，**PlayerController** 内部需要追加 `soundManager->playGunSound()` 和 `statsRecorder->recordShot()`。每个事件源都直接依赖所有事件消费者，依赖方向会变得不可预测。

更关键的是，这种直接调用方式意味着：**模块 A 必须知道模块 B 的完整接口**。如果将来把 **Widget** 换成另一个渲染器（比如从 **QWidget** 迁移到 **QML**），或者给 **CombatManager** 增加一个日志记录器，所有调用方都需要修改。这在只有 3-4 个模块时还能忍受——本项目 10+ 模块间的信息流如果全部走直接调用，修改任何一个消费者接口都会引发连锁修改。

3.2 技术方案与设计思路

解决上述问题的核心思路是**观察者模式**：事件源不直接调用消费者，而是“广播”一个信号出去，由消费者自己决定是否订阅。这种模式在不同平台有不同的实现——Qt 有 `signal/slot`，Boost 有 `signals2`，C# 有 `delegate/event`，甚至 C 语言也能通过函数指针回调列表手动实现。无论哪种实现，核心都是把“谁发出事件”和“谁来响应事件”解耦。

3.2.1 信号槽在本项目中的角色

在 Qt 框架中，信号槽是语言级别的观察者模式实现：任何继承 **QObject** 的类都可以声明 **signals**（信号）并通过 `emit` 广播事件，`connect()` 负责建立信号到槽函数（或 `lambda`）的绑定。

本项目中，信号槽承担了三类通信角色：

第一类：驱动帧循环的定时信号。这是最高频的信号——每 16ms（约 60FPS）`QTimer::timeout` 触发 `Widget::gameLoop()`，整个游戏的“输入→逻辑→渲染”都在这个槽函数内顺序执行。信号在这里解决的是一个“周期性触发”的需求——如果用死循环加延时来做帧循环，Qt 的事件处理（包括接收键盘输入、重绘 UI）会被阻塞。

```
// widget.cpp 构造函数
timer = new QTimer(this);
connect(timer, &QTimer::timeout, this, &Widget::gameLoop);
// 启动后每 16ms 自动触发一次 gameLoop()
timer->start(16);
```

第二类：跨模块事件通知。`PlayerController` 在检测到攻击指令时，不创建任何实体，只发出 `requestThrowBall` 信号并传参——调用方只需提供“谁在什么位置以什么速度发射了什么武器”。`Widget` 在连接这个信号后负责创建对应的实体对象。`PlayerController` 完全不知道 `Entity` 的存在。

```
// playercontroller.h —— 信号声明
signals:
    void requestThrowBall(float x, float y, float vx, float vy,
                          Player::WeaponType weapon, float damage,
                          float frozenBonus, int id);

// playercontroller.cpp —— 攻击时发出信号（节选）
if (player.lock()->weapon == Player::WeaponType::ball) {
    emit requestThrowBall(player.lock()->x, player.lock()->y,
                          player.lock()->vx, player.lock()->vy,
                          Player::WeaponType::ball,
                          getAttackDamage(), ...);
}
```

```
// widget.cpp —— 在游戏初始化时建立连接
connect(c1.get(), &PlayerController::requestThrowBall,
        this, &Widget::onPlayerRequest);
connect(c2.get(), &PlayerController::requestThrowBall,
        this, &Widget::onPlayerRequest);
```

这样一来，**PlayerController** 和实体创建逻辑是完全隔离的。`PlayerController` 只负责“报告攻击请求”，如果将来需要给攻击添加屏幕震动或手柄振动，直接在 `Widget` 的 `onPlayerRequest` 槽函数中添加逻辑即可，`PlayerController` 不需要改一行代码。

第三类：页面间导航和状态同步。游戏的页面流是 `Setup` → `Game` → `Result`，每次页面切换都涉及跨模块信息传递。例如 `SetupWidget` 完成设置后发出 `sessionReady(session)`，`MainWindow` 接收后隐藏设置页、创建游戏页并启动对局：

```
// mainwindow.cpp
connect(setupPage, &SetupWidget::sessionReady,
        this, &MainWindow::startGame);
```

3.2.2 动态连接与断开

信号的另一个重要特性是可以在运行时动态连接和断开。本项目中广泛使用了这个能力——当游戏结束时断开定时器信号以停止帧循环，当结算面板关闭时断开其信号以防止悬空引用：

```
// 游戏结束，停止定时器
disconnect(timer, nullptr, this, nullptr);
// 结算面板关闭时断开其信号
disconnect(endlessResultWidget, nullptr, this, nullptr);
```

以及用临时连接处理一次性事件——ModifierOverlay 弹出后，连接一次 `keyPressed` 信号，用户按任意键后立即断开：

```
connect(this, &Widget::keyPressed, overlay, [=]() {
    disconnect(this, &Widget::keyPressed, overlay, nullptr);
    // 处理按键 ...
});
```

这种“接上-用过-断开”的模式避免了长期连接的累积，也防止了已经销毁的对象仍在接收信号导致崩溃。

3.2.3 并非所有通信都用信号

项目并没有把所有模块间通信都交给信号槽。CombatManager 的战斗结算、SoundManager 的音效播放、Map 的碰撞查询，都通过直接函数调用完成。原因在于：

- CombatManager 的结算函数在 `gameLoop` 中的执行时机是固定的（“先移动、后碰撞、最后结算”），消费者明确且唯一。信号槽的订阅/广播机制在这里是过度设计。
- SoundManager 的调用点分散在 CombatManager、PlayerController、Widget 三个模块中，但它是一个只读的全局服务——调用方自己决定什么时候播什么音效，不需要 SoundManager 反向通知调用方任何事。

这个区分的依据是：信号槽适用于“一对多”和“未来可能有新消费者”的场景，而直接调用适用于“一对一”和“时机固定”的场景。把信号槽用在该用的地方，而不是不加区分地全部走信号槽。

3.3 代码验证

验证项	输入/触发	预期行为	结论
发射信号-实体创建链路	玩家按键近战/投掷/射击，AI 触发攻击	PlayerController 发出 requestThrowBall, Widget::onPlayerRequest 创建对应 Entity 并加入场景	发 通过
定时信号驱动帧循环	游戏运行中	QTimer::timeout → Widget::gameLoop 每 16ms 自动触发，帧率稳定在约 60FPS	通过
页面导航信号链	主菜单→ 设置→ 对局→ 结算→ 返回菜单	SetupWidget::sessionReady → MainWindow::startGame; Widget::matchResultConfirmed → MainWindow::showMenu, 信号驱动的页面切换全程无需模块间相互持有指针	通过
动态断连防悬空	对局结束，定时器停止	disconnect(timer, nullptr, ...) 后 gameLoop 不再被触发，无悬空引用或崩溃	通过
词条选择信号	无尽模式每轮结束	ModifierOverlay 弹出三选一，用户点击 optionChosen → Widget::onModifierChosen 应用词条到玩家，面板关闭	通过

3.4 替代方案分析

方案	优势	代价/为何未采用
手动回调注册（函数指针或 std::function 列表）	无框架依赖；完全透明可见	需要手动管理回调列表（注册、注销、调用）；类型安全性靠程序员保证；链接错误不易排查。相当于自己实现一个粗糙版信号槽，维护成本高

事件总线 / 消息队列	全局消息通道，任何模块可发布和订阅；支持消息过滤和优先级；天然支持跨线程	引入额外的中间层和消息类型枚举；小型项目中增加复杂度多于收益；调试时消息路由不透明，难以追踪“谁响应了这个事件”
全部使用直接函数调用	调用关系显式、IDE 可直接跳转、性能最优	每个模块都要持有所有可能消费者的指针或引用；依赖方向复杂；新加消费者需要改事件源代码。本项目的规模（10+ 模块）已经到了这个方案的维护边界
Qt 信号槽（当前方案）	观察者模式的语言级支持；类型安全（编译期检查信号与槽签名匹配）；支持动态连接/断开；Qt 框架内置，无需额外库	依赖 Qt 的 moc 预编译器和 Q_OBJECT 宏；信号槽调用有轻微间接开销（对 60FPS 游戏可忽略）。本项目已使用 Qt 框架，无需额外引入依赖

3.5 小结

信号槽在本项目中不是一个“用了 Qt 就顺便用一下”的功能，而是从模块间通信的复杂度出发推导出的必要方案。项目的实际使用方式也体现了选择性——高频固定的调用（如 CombatManager 结算）仍然走直接调用，只有“一对多”“动态订阅”“跨层级通知”的场景才用信号槽。这种“什么场景用什么通信方式”的判断，比“统一用信号槽”或“坚决不用信号槽”都更有价值。

4 技术三：数据驱动的游戏玩法可扩展性

4.1 游戏中引出的问题

格斗游戏需要“养成感”和“变化性”来支撑重复可玩性——尤其在无尽模式中，每一轮结束玩家从三个随机词条中选择一个，词条效果叠加，逐步变得更强；同时 AI 对手的强度也在递增。本项目的词条和法术系统加起来涉及：

- **57 种词条类型**（13 种通用 + 44 种法术专属），全部在 `ModifierType` 枚举和工厂方法中定义，其中已实现法术（定身/隐身/铜头铁臂）的约 25 种可在游戏中获取
- **3 种法术**（定身、隐身、铜头铁臂），各有不同的激活条件、持续时间和交互规则
- **30+ 个词条效果字段**集中在 `Player::ModifierState` 结构体中——移速倍率、各武器伤害倍率、减伤率、法术专属加成等
- **多层乘区叠加**：词条之间按独立乘区结算，同词条多级线性叠加

如果把这些全部硬编码进战斗代码，`getAttackDamage()` 会变成什么样子？每加一个词条就要加一个 if 分支，每加一个法术就要在伤害计算中插入一段特殊逻辑。57 种词条 × 5 种武器 × 多种法术互动的排列组合将产生数百个需要维护的条件分支。

更关键的是可修改性问题：如果要调整“把定身增伤从 20% 改成 25%”，只需改 `modifierdata.h` 中工厂方法里一个数值。如果全部硬编码，需要找到伤害计算函数中某一行埋在 500 行代码里的常量，还不敢保证改的是对的。

4.2 技术方案与设计思路

核心思路是**数据与逻辑分离**：用结构体描述“词条是什么”，玩家状态字段存储“词条产生了什么效果”，战斗逻辑代码**遍历**状态字段来结算——战斗逻辑本身不包含任何词条的细节。

4.2.1 三个层次的数据驱动设计

第一层：词条数据的定义（`ModifierData`）。文件 `modifierdata.h` 将每类词条封装为一个 `ModifierData` 结构：

```
struct ModifierData {
    ModifierType type;
    QString      displayName;    // 卡片标题，如"急转轮回"
    QString      description;    // 卡片描述，如"法术冷却 -5秒"
    int          level = 1;
};

enum class ModifierType {
    SPELL_COOLDOWN_REDUCE, MAX_HP_UP, ENEMY_MAX_HP_DOWN,
```

```

MOVE_SPEED_UP, ENEMY_MOVE_SPEED_DOWN,
PUNCH_DAMAGE_UP, KNIFE_DAMAGE_UP, BALL_DAMAGE_UP, GUN_DAMAGE_UP,
DAMAGE_BONUS_UP, DAMAGE_REDUCTION_UP,
DOUBLE_EDGE, EXTRA_WEAPON_SLOT,
FREEZE_DURATION_UP, FREEZE_DAMAGE_UP, FREEZE_BREAK_CDR,
STEALTH_DURATION_UP, STEALTH_SPEED_UP, STEALTH_REGEN, STEALTH_FIRST_HIT,
// ... 其余 30+ 种
};

```

工厂方法 `poolForPlayer(spellName)` 根据玩家的法术类型返回合法的词条池——通用词条 + 法术专属词条。例如定身法玩家会看到“急转轮回”“长缚”“破而后立”等，不会看到隐身专属的“破影一击”。

// 工厂方法示例：按法术返回专属词条

```

inline QVector<ModifierData> ModifierData::forSpell(const QString& spellName) {
    if (spellName == "freeze") {
        return {
            { FREEZE_DURATION_UP, "长缚", "定身持续时间 +1.5秒" },
            { FREEZE_DAMAGE_UP, "缚中伤", "定身期间伤害 +20%" },
            { FREEZE_BREAK_CDR, "破而后立", "定身破碎后冷却 -5秒（唯一）" },
        };
    }
    if (spellName == "stealth") {
        return { /* 隐身专属词条 */ };
    }
    // ...
}

```

第二层：玩家状态字段（`ModifierState`）。Player 内部维护一个 `ModifierState modifiers` 结构体，包含所有词条生效后的数值累加结果：

```

struct ModifierState {
    int    spellCooldownReduce    = 0;
    float  maxHpMultiplier        = 1.0f;
    float  moveSpeedMultiplier    = 1.0f;
    float  punchDmgMultiplier     = 1.0f;
    float  knifeDmgMultiplier     = 1.0f;
    float  ballDmgMultiplier      = 1.0f;
    float  gunDmgMultiplier       = 1.0f;
    float  damageBonusMultiplier  = 1.0f;
    float  damageReduction        = 0.0f;
    bool   doubleEdge             = false;
}

```

```

float freezeDurationBonus    = 0.f;
float freezeDmgMultiplier    = 1.0f;
bool freezeBreakCDR          = false;
// ... 30+ fields
};

```

第三层：词条应用到状态字段（**applyModifier**）。当玩家拾取词条时，`Player::applyModifier(const ModifierData& mod)` 根据 `mod.type` 将数值累加到对应的状态字段：

```

void Player::applyModifier(const ModifierData& mod) {
    switch (mod.type) {
    case ModifierType::SPELL_COOLDOWN_REDUCE:
        modifiers.spellCooldownReduce += 5;
        spellState.cooldownMax = qMax(5.f, spellState.cooldownMax - 5.f);
        spellState.cooldownRemain = qMax(0.f, spellState.cooldownRemain - 5.f);
        break;
    case ModifierType::DAMAGE_BONUS_UP:
        modifiers.damageBonusMultiplier += 0.25f; break;
    case ModifierType::DAMAGE_REDUCTION_UP:
        modifiers.damageReduction = qMin(0.6f, modifiers.damageReduction + 0.2f);
        break;
    case ModifierType::DOUBLE_EDGE:
        modifiers.doubleEdge = true; break;
    // ... 50+ cases
    }
}

```

这个 `switch` 是整个系统中唯一需要了解词条类型到状态字段对应关系的地方。战斗代码完全不碰 `ModifierType` 枚举。

战斗代码消费的是状态字段，不是词条类型。`getAttackDamage()` 展示的就是这一层：

```

float Player::getAttackDamage() {
    float base = 0.f;
    switch (weapon) {
    case WeaponType::punch: base = 5.f * modifiers.punchDmgMultiplier; break;
    case WeaponType::knife: base = 15.f * modifiers.knifeDmgMultiplier; break;
    case WeaponType::ball:  base = 1.f * modifiers.ballDmgMultiplier; break;
    // ...
    }
    base *= modifiers.damageBonusMultiplier;
    if (modifiers.doubleEdge) base *= 1.5f;
    // 隐身破影一击有额外的随时间增长伤害倍率
}

```

```

    if (spellState.stealthActive && modifiers.stealthFirstHit && ...) {
        float ratio = qMin(elapsed / totalDuration, 1.0f);
        base *= (1.0f + ratio);
    }
    return base;
}

```

注意：getAttackDamage() 不知道“伤害+20%”是哪来的——它只看到 damageBonusMultiplier 字段当前值是 1.25（吃了一个词条），它无法也不需区分这是哪个词条类型贡献的。这就是数据驱动的要點：战斗逻辑对数据来源保持无知。

4.2.2 SpellState：法术状态的纯数据结构

法术相关的运行时状态也设计为纯数据结构（spellstate.h），不含任何业务逻辑：

```

struct SpellState {
    float cooldownRemain = 0.f;    // 剩余冷却时间
    float cooldownMax    = 0.f;
    bool  isFrozen       = false; // 是否被定身
    float frozenRemain   = 0.f;
    bool  stealthActive  = false; // 隐身是否激活
    float stealthRemain  = 0.f;
    bool  ironBodyActive = false; // 铜头铁臂是否激活
    // ... 其他法术状态

    void tick(float dt) {
        // 所有定时器统一递减，到零自动关闭状态
        if (cooldownRemain > 0.f) cooldownRemain -= dt;
        if (frozenRemain > 0.f) { frozenRemain -= dt;
            if (frozenRemain <= 0.f) isFrozen = false; }
        // ...
    }
};

```

注释：tick() 中直接写了递减逻辑，未抽象为虚函数。原因是所有法术计时器共享完全相同的行为（递减→触零→关闭标志），抽象为虚函数只会增加代码跳转次数而不减少任何重复。这是数据驱动设计中的一个务实判断：当数据的“结构”一样时，不值得为结构相同的类型引入多态。

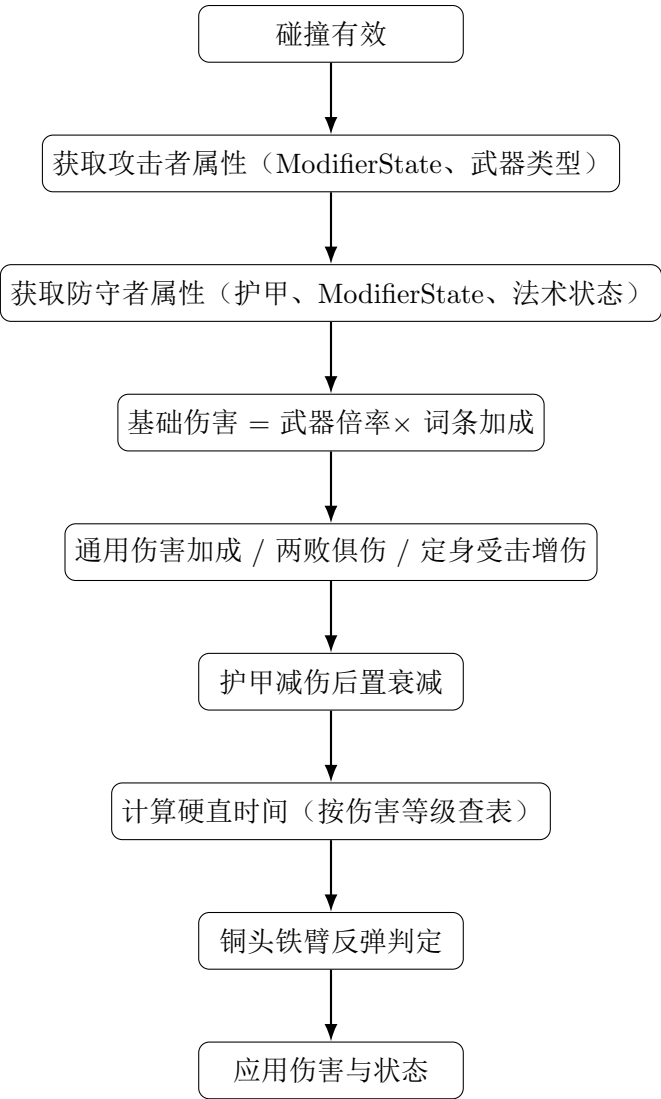


图 2: 伤害计算流水线。注意：每一步消费的都是 ModifierState 中的数值字段，而非 ModifierType 枚举——这正是技术三“数据驱动”的关键：战斗代码对词条来源保持无知。

4.3 代码验证

验证项	输入/触发	预期行为	结论
词条池生成	定身法玩家进入无尽模式	poolForPlayer("freeze") 通过 返回 13 通用 + 3 法术专 属，不包含其他法术的专 属词条	通过
词条叠加	连续拾取两次“伤害加成+25%”	ModifierState.damageBonusMultiplier = 1.5, getAttackDamage 返回伤害 = 基础 × 1.5	通过

验证项	输入/触发	预期行为	结论
唯一词条	尝试多次获取“两败俱伤”	isUniqueModifier 返回 true，UI 层阻止重复选择	通过
新词条零影响战斗代码	新增“护甲穿透”词条	只需在 enum 加一项 + applyModifier 加一个 case + ModifierState 加一个字段；getAttackDamage 和 CombatManager 完全不改	通过
法术定时器统一 tick	冻结、隐身、铜头同时激活	SpellState::tick(dt) 统一递减所有计时器，到期自动关闭对应标志	通过

4.4 替代方案分析

方案	优势	代价/为何未采用
全部硬编码（if-else 分支）	直接明了，IDE 可跳转	57 种词条×多处消费点 = 代码爆炸；新增词条需要改动 combat + UI + 记录等所有涉及模块
脚本化（Lua/Python 嵌入）	策划可独立修改数值和行为逻辑而不需 C++ 编译	需要集成脚本解释器和 C++ 脚本绑定层；调试变复杂（C++ 调用栈 + 脚本调用栈混合）；课程周期内投入产出比低
JSON/配置表驱动	词条数据完全外部化（文本文件），非程序员可编辑	需要解析器和错误处理；加载失败时的降级策略需额外设计；运行时类型安全不如编译期 enum
当前方案（enum + struct + switch）	编译期类型检查；新增词条只需改一个 switch；战斗代码对来源无知；无外部依赖	词条数量达到 100+ 时，applyModifier 的 switch 会变长；但 57 种量级完全可控

4.5 小结

本项目的词条与法术系统通过三层数据驱动架构（定义层 ModifierData → 状态层 ModifierState → 消费层 getAttackDamage）实现了可扩展性。关键设计决策是不让战斗代码知道词条类型——战斗逻辑只读状态字段，不关心字段值从哪来。这个决策使“新增一种词条”的成本从“修改多个模块”降为“修改一个 enum 和一个 switch case”。

5 技术四：有限状态机驱动的 NPC 决策

5.1 游戏中引出的问题

单局对战模式中，AI 需要扮演一个“像人”的对手。一个简单的跟随+攻击脚本——“看到玩家就冲过去打”——在静态环境中或许勉强可用，但这个游戏中有如下变量：

- 玩家可能隐身后绕到侧面偷袭，AI 需要处理“看不见目标”的情况
- 地面可能刷新更好的武器或词条，AI 需要判断“是继续打还是先去捡”
- AI 血量低时需要考虑撤退、找补给或换策略
- 地图上移动路径可能被阻隔（平台间需要寻路跳转），也可能卡在平台边缘反复抖动
- 释放法术后应该有一段“乘胜追击”的窗口，而不是放完技能立刻摆烂
- AI 不能太强（玩家感受极差），也不能太弱（无聊），需要对不同水平的玩家有区分度

这些问题无法用一串 if-else 解决——决策条件会指数级交叉（“血量低 AND 对手隐身 AND 地上有狙击枪 AND 我正在卡住→怎么办？”）。需要一个结构化的决策框架。

5.2 技术方案与设计思路

本项目的 AI 采用了分层的有限状态机 + 特殊分支优先的架构。核心思想是：常规战术行为交给状态机轮流执行，但紧急或特殊事件（隐身、卡住、施法后追击）直接跳出状态机走独立处理路径。

5.2.1 三层决策架构

// gameai.cpp — updateIntent() 的主流程（简化版）

```
void GameAI::updateIntent(MoveIntent& moveIntent, AttackIntent& attackIntent) {
    moveIntent = MoveIntent::NONE; attackIntent = false;

    // —— 第一层：特殊策略（优先级最高，跳过状态机） ——
    if (隐身 + 破影一击) { executeStealthFirstHitStrategy(...); return; }
    if (!目标可见)         { handleStealthTarget(...);           return; }
    if (卡住了)             { handleStuckSituation(moveIntent);   return; }
    if (正在执行动作计划) { 弹出下一帧动作, return;              }

    // —— 第二层：状态机执行 ——
    switch (m_currentState) {
    case AIState::FOLLOW:      executeFollow(moveIntent, ...); break;
    case AIState::ATTACK:      executeAttack(moveIntent, ...); break;
    case AIState::SEEK_WEAPON: executeSeekWeapon(...);         break;
    // ... 共 8 个状态
```

```

    }

    // —— 第三层：状态转换 ——
    AIState nextState = determineNextState();
    if (nextState != m_currentState) m_currentState = nextState;
}

```

这个结构清晰地划分了三个优先层级：

1. **特殊策略层**：“活下来”的需求优先于“打得聪明”。隐身、卡住、执行预定计划等情况直接劫持流程，不经过状态机。这避免了状态机在异常情况下做出不合理的切换（比如隐身时 AI 跑去“找武器”而不是搜索最后已知位置）。
2. **状态机层**：在正常情况下，由当前状态对应的执行函数生成 MoveIntent 和 AttackIntent。每个状态只关心“当前状态下我该做什么”——FOLLOW 负责寻路接近目标，ATTACK 负责判定距离并发动攻击，SEEK_WEAPON 负责朝最优武器掉落位置移动。
3. **状态转换层**：determineNextState() 在每帧末尾评估全局局势，决定是否切换状态。转换评估与执行是分离的——当前状态的执行函数不关心“下一帧会不会换状态”。

5.2.2 状态转换的决策逻辑

determineNextState() 是本系统的核心（约 85 行），综合评估多项条件来决定状态切换：

- **是否无法伤害对手 (canDamageTarget)**：拳击打链甲无效、弹药耗尽等情况 → 优先找武器
- **地面上是否有更好的物资 (hasWeaponDrop / hasArmorDrop / hasSupplyDrop)** → 降低攻击倾向，提升搜索倾向
- **有无尽模式**：无尽模式中移除了“低血量撤退”逻辑（因为 AI 有 endlessImmortal 可复活），改为纯概率驱动
- **血量比例 (healthRatio ≤ 32%)**：触发人格分化——“背水一战”人格继续打，“游走机动”人格撤退找药
- **长期目标锁 (GOAL_LOCK_DURATION)**：每约 10 秒通过 refreshStrategicGoal() 重新抛骰子决定“进攻为主”还是“搜刮为主”，锁定期内不频繁切换，避免帧间抖动

人格系统为决策注入了可控的随机性。AI 出生时随机分配两种人格之一：

- **BACK_TO_WALL (背水一战)**：低血量时仍倾向进攻，搜刮倾向低（25%-35%）
- **KITE_SCAVENGER (游走机动)**：低血量时大幅提升搜刮概率（60%-80%），优先找补给和装备

人格设计的目的不是让 AI “聪明”，而是让每次对局的 AI 行为不可预测。相同配置下，不同人格的 AI 会给玩家完全不同的感觉。

5.2.3 特殊分支的实际价值

卡住检测与脱困是特殊分支中最直接体现工程价值的。AI 在复杂平台地形中可能卡在边缘反复左右晃动。检测方式：连续多帧位移小于阈值→ 判为卡住→ 清空动作计划→ 执行脱困跳跃。这个逻辑完全跳过状态机，无论当前是什么状态，都能第一时间响应，极大提升了 AI 的流畅度和玩家的体验。

施法后强制攻击模式解决了一个微妙的时序问题：AI 释放法术后，如果下一帧 `determineNextState()` 评估为“距离太远要跟随”，法术的战略价值就白费了。因此在 `executeFollow()` 内部插入了一段施法后的强制逻辑——法术释放后的 60 帧内强制保持攻击态势，靠近对手并连续攻击。

隐身目标处理则展示了状态机覆盖不到的边缘情况。玩家隐身时，AI 不再用状态机跟踪目标，而是前往玩家隐身前的最后已知位置巡逻。如果AI带“破影一击”词条，AI 还有专门的前期积攒物资、最后时机出手的策略。

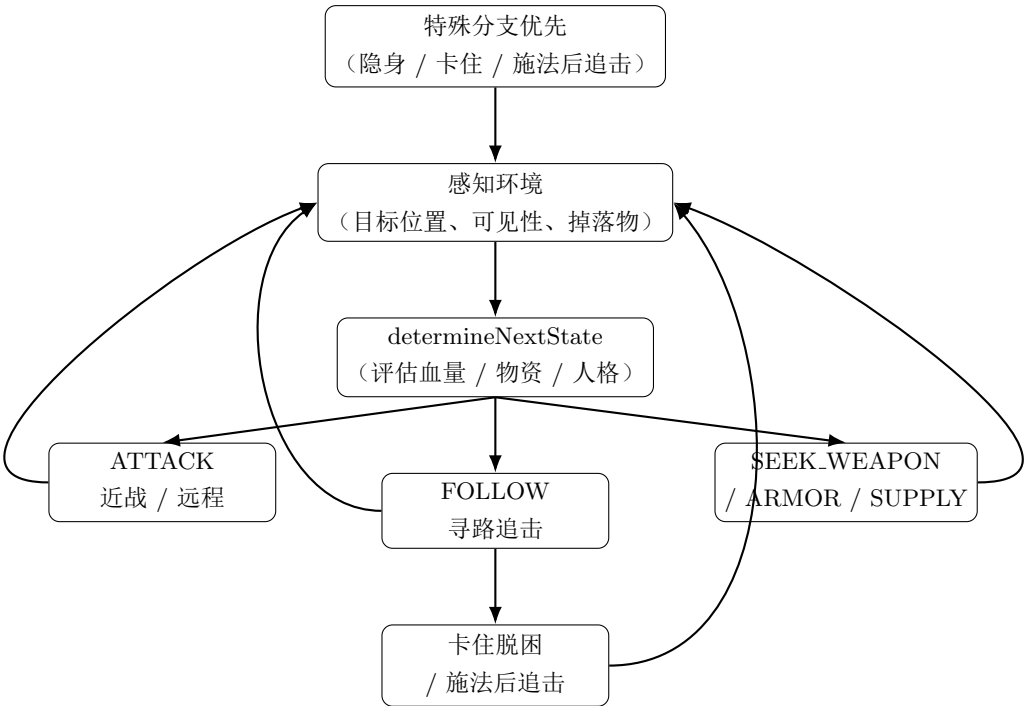


图 3: AI 决策流程。顶层特殊分支（隐身/卡住/施法后追击）直接跳过状态机；正常流程走感知→ 状态转换评估→ 状态执行；卡住和施法后强制模式为独立路径，不受状态机当前状态影响。

5.3 代码验证

验证项	输入/触发	预期行为	结论
FOLLOW 状态寻路	玩家在地图另一端	AI 通过 Map BFS 算出平台序列，生成朝向玩家的移动意图，遇到平台边缘自动跳跃	通过

状态切换	AI 血量降至 30% + 人格为 KITE_SCAVENGER	determineNextState 返回 RETREAT 或 SEEK_SUPPLY，AI 从攻击转为撤退/搜刮	通过
卡住脱困	AI 卡在平台边缘连续 100 帧位移；5 像素	isStuck() 返回 true → handleStuckSituation 清空动作计划并执行跳跃脱困	通过
施法后追击	AI 释放定身法命中玩家	进入 60 帧强制攻击窗口，AI 朝被定身的玩家移动并持续输出，不会切回 FOLLOW	通过
隐身搜索	玩家激活隐身	AI 保存最后已知位置，前往该位置附近巡逻，不会呆立原地	通过
人格分化	同一配置对局两局	两局 AI 人格可能不同（出生时随机），导致低血量时行为截然不同	通过

5.4 替代方案分析

方案	优势	代价/为何未采用
纯 if-else 决策树	无框架开销，直接	条件组合爆炸（血量× 可见性× 物资× 地形），维护不可持续
行为树（Behavior Tree）	模块化决策节点，易于可视化和组合；Unreal Engine 等引擎内置支持	实现一个可用的行为树框架本身就需要大量代码（节点基类、组合节点、装饰器、黑板）；对 C++ 课程项目来说，实现行为树的成本已超过调试 AI 行为的成本
GOAP/规划器	最优动作序列搜索，理论上能产生更智能的行为	实现极其复杂，且规划结果不可预测，调试困难。对于 8 状态的小型状态机，规划器过度设计
当前方案（分层 FSM）	实现成本低；状态边界清晰，出问题能快速定位是“特殊分支”“状态机”还是“状态转换”的 bug；预留了 8 个状态的扩展空间	状态增多时转换矩阵变复杂；相较行为树，表达并行行为（一边移动一边射击）不够自然——本项目中通过“施法后强制模式”这类特殊分支补偿了这个弱点

5.5 小结

有限状态机在本项目中不是“最先进的 AI 技术”，但它足够胜任 8 个状态 + 4 个特殊分支的决策空间，且在课程周期内能以最低的代码量提供可区分的 AI 行为。关键设计是**特殊分支跳过状态机**和**目标锁定防抖动**——前者处理紧急情况，后者防止决策在相邻帧之间反复切换。加上人格系统注入的随机性，FSM 驱动的 AI 在有限的复杂度下交出了“不可预测且可维护”的答卷。

6 技术五：实时游戏主循环的时序设计

6.1 游戏中引出的问题

格斗游戏的实时性要求极高。一帧 16ms 内要完成：读取键盘输入→ AI 决策→ 移动角色→ 碰撞检测→ 伤害结算→ 生成/清理实体→ 播放音效→ 重绘屏幕。这些子系统之间存在严格的时序依赖：

- 碰撞检测**必须**在移动之后执行——否则检测的是上一帧的位置，“子弹穿透角色”会频繁出现
- 死亡检查**必须**在伤害结算之后——结算前检查会导致“最后一击”被跳过
- 实体清理**必须**在所有计算完成后——提前清理会让还在空中的子弹在被命中判定前消失
- 帧时间 dt 必须基于真实耗时——如果机器卡顿了一帧（比如 0.1s 而不是 0.016s），角色的移动距离和冷却计时需要同步放大，否则会出现“时间变慢”的观感

简单地把所有模块扔进一个 while 循环而不管理执行顺序，会导致各种难以复现的 bug：子弹偶尔穿透、死亡触发时机不一致、卡顿后行为异常。这些 bug 的共同特征是不稳定——同样的攻击有时命中有时穿透，给调 bug 带来极大痛苦。

6.2 技术方案与设计思路

6.2.1 固定的帧更新流水线

本项目的 gameLoop 函数约 40 行，但其中包含了一个严格顺序的流水线。下面展示完整流程（注释中标记了每个步骤的目的和时序依赖）：

```
void Widget::gameLoop() {
    // —— 1. 计算真实帧时间（处理卡顿补偿） ——
    float dt = lastTime.restart() / 1000.0f;
    m_gameTime += dt;

    // —— 2. 计时器与生成逻辑 ——
    // 无尽模式 AI 定时获取词条、随机生成掉落物
    if (currentSession.mode == GameSession::Mode::ENDLESS) {
        m_endlessAiModifierTimer += dt;
        if (m_endlessAiModifierTimer >= ENDLESS_AI_MODIFIER_INTERVAL)
            applyRandomModifierToAI();
    }
    spawnDrop();           // 每帧尝试生成掉落物
    updateDrops(dt);       // 更新已有掉落物计时

    // —— 3. 死亡检查（伤害结算之前不能做） ——
    if (players[0]->isDead()) { gameEnd(1); return; }
```



```

    if (players[1]->isDead()) { gameEnd(0); return; }

    players[0]->setDt(dt); players[1]->setDt(dt);

    // —— 4. AI 决策（生成意图，不执行移动） ——
    ai->updateIntent(moveIntent, attackIntent);
    intent[1].moveIntent    = moveIntent;
    intent[1].attackIntent  = attackIntent;

    // —— 5. 执行移动（玩家 + AI） ——
    controllers[0]->handleIntent(intent[0].moveIntent, ...);
    controllers[1]->handleIntent(intent[1].moveIntent, ...);

    // —— 6. 碰撞检测与战斗结算 ——
    //      位置已更新，此时碰撞检测的是"最新位置"
    cm.checkPlayerVsPlayerCollision(controllers[0], controllers[1]);
    updateBalls(dt);    // 飞行物移动 + 碰撞检测

    // —— 7. 状态更新 ——
    tickSpells(dt);    // 法术冷却递减

    // —— 8. 清理过期对象（所有结算完成后） ——
    entities.erase(std::remove_if(entities.begin(), entities.end(),
                                   [](auto& e) { return e->shouldBeRemoved(); }),
                   entities.end());

    // —— 9. 触发重绘（paintEvent） ——
    update();
}

```

这个时序设计的核心原则是：**信息在帧内单向流动，不做跨步骤回写**。步骤 1-3 收集信息，步骤 4-5 更新位置，步骤 6 基于最新位置结算，步骤 7-9 做收尾和展示。如果步骤 6 修改了某个实体的位置（比如反弹），被反弹的实体位置变化会在下一帧的步骤 5 中生效——不会出现“同一帧内两个步骤并发修改导致不一致”的问题。

6.2.2 帧时间补偿（dt 传递）

卡顿是不可避免的——垃圾回收、操作系统调度、磁盘 IO 都可能阻塞一帧。本项目中 dt 的传递路径是：

```
lastTime.restart() → float dt → players[i]->setDt(dt) → Entity::setDt(dt) → 物理公式
```

在 `applyGravity()` 中，位置更新用的是 $v_y = v_y + g \cdot dt$ 而非 $v_y = v_y + g \cdot 0.016$ 。如果 $dt = 0.1s$ （卡顿了 6 帧），角色会跳跃 $0.1/0.016 \approx 6$ 倍的距离，补偿了丢失的时间。这个方法叫可变时间步长（variable timestep），优点是实现简单且卡顿后位置一致性不丢失，缺点是物理模拟不是严格确定性的（两次运行因 dt 微小差异可能产生略有不同的轨迹）。对本项目这种不要求帧精确复现的格斗游戏，这个代价是可接受的。

6.2.3 对象生命周期管理

球和子弹是瞬态对象（击中后立即删除），但删除时机不当会导致悬挂指针或 double-free。本项目使用 `std::shared_ptr<Entity>` 统一管理实体对象，配合 `std::weak_ptr` 在掉落物和球容器中存储弱引用：

```
std::vector<std::shared_ptr<Entity>> entities; // 所有实体（拥有所有权）
std::vector<std::weak_ptr<Ball>> balls;      // 球引用（不拥有所有权）
std::vector<std::weak_ptr<DropItem>> drops;  // 掉落物引用（不拥有所有权）
```

每帧末尾通过 `remove_if + shouldBeRemoved()` 清理 `entities` 中的过期对象，同时用 `weak_ptr::expired()` 清理 `balls` 和 `drops` 中已失效的引用。这个机制的优点是：`balls` 容器中的引用在 `entity` 被销毁后自动变空，不会出现“拿到指针但对象已经被删除”的情况。

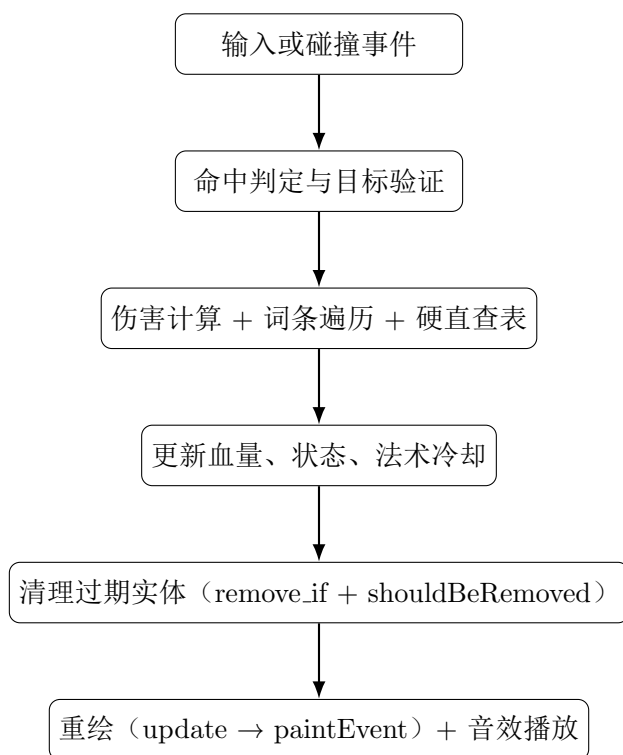


图 4：单帧执行流水线。箭头表示信息流动方向——前一步的输出是后一步的输入。所有步骤在当前帧内顺序执行，不跨步骤回写，保证每个步骤操作的数据在帧内是一致的。

6.3 代码验证

验证项	输入/触发	预期行为	结论
碰撞时序	玩家和 AI 同时朝对方冲刺并出拳	碰撞在双方移动后结算，通过命中判定基于最新位置，不会出现“打空气命中”或“穿透”	
卡顿补偿	模拟帧卡顿（强制 sleep 50ms）	dt 变大，角色跳跃高度和移动距离同步放大，不出现“画面停顿但逻辑加速”的现象	通过
对象安全清理	投掷球飞行 3 秒后自动消失	entities 末尾 remove_if 清理，balls 中的 weak_ptr::expired 返回 true，同步清理弱引用	通过
死亡检查时序	双方同时对最后一击	伤害结算完成后才检查 is-Dead，不存在“击杀了但伤害没扣”的情况	通过

6.4 替代方案分析

方案	优势	代价/为何未采用
固定时间步长（Fixed Timestep）	物理模拟严格确定性，回放和录像可精确复现	需要累积器（accumulator）机制，额外的代码复杂度；慢帧会导致“螺旋式死亡”（多帧积压同时结算）。本项目的精度需求不要求帧精确复现
多线程游戏循环	可利用多核并行处理物理/AI/渲染	线程同步会极大增加复杂度（锁、原子操作、数据竞争）；对于 60FPS 的 2D 游戏，单线程已足够支撑所有计算
当前方案（单线程可变 dt 流水线）	实现极简，时序完全可控，调试路径清晰	物理非确定性的代价；如果未来加入联网对战或录像回放，需要改为固定时间步长

6.5 小结

游戏主循环的时序设计解决的是一个信息流编排问题：在每 16ms 的时间窗口内，如何安排子系统执行顺序使得每步计算都基于“最新且一致”的信息。本项目的方案选择了最简单可维护的路径——单线程、固定顺序、可变时间步长——在 60FPS 要求下稳定运行，且调试时卡在某一步可以精确知道哪些数据已经更新、哪些还没变。

7 技术六：跨平台构建系统的抽象与配置

7.1 游戏中引出的问题

这个游戏的源代码组织面临一个“纯代码之外”的工程问题。项目包含：

- 40+ 个源文件（.h / .cpp），分布在 10 个模块中
- 依赖 Qt 6 的三个独立模块：Widgets、Multimedia、Core
- 一个 Qt 资源文件（.qrc）管理图片和音效资源
- 需要在 Windows（MinGW）和 macOS 上都能编译
- 使用 C++17 标准（结构化绑定、`std::optional`、`if constexpr` 等特性）
- MSVC 编译器需要处理 UTF-8 中文注释的编码

如果手动管理编译，典型的一条 g++ 编译命令会是：

```
g++ -std=c++17 -I/path/to/Qt6/include -I/path/to/Qt6/include/QtWidgets
-I/path/to/Qt6/include/QtCore -I/path/to/Qt6/include/QtMultimedia
-L/path/to/Qt6/lib -lQt6Widgets -lQt6Core -lQt6Multimedia
main.cpp widget.cpp player.cpp ... (30+ 个文件)
-o shooting_game
```

这条命令超过 300 个字符，而且：

- 在 Windows 上 Qt 的路径是 `C:\Qt\6.5.3\mingw_64\include`，macOS 上是 `/usr/local/opt/qt@6/include`——路径完全不同
- MinGW、MSVC、Clang 的编译选项名称不同（比如 C++17 标准在 MSVC 是 `/std:c++17` 而非 `-std=c++17`）
- 头文件依赖关系改变后（比如 `player.h` 新增了 `#include "spellstate.h"`），需要知道哪些 .cpp 需要重新编译，只手动重编改动的文件是不现实的

7.2 技术方案与设计思路

CMake 是一个构建系统生成器——它读取平台无关的 `CMakeLists.txt`，生成平台原生的构建文件（Windows 上生成 Ninja 或 Visual Studio 项目，macOS/Linux 上生成 Makefile 或 Ninja），再由原生工具执行实际编译。CMake 本身不编译任何代码，它做的事情是把“我要做什么”翻译成“这个平台上怎么做”。

7.2.1 CMake 如何解决本项目的问题

第一：平台无关的编译器配置。

```
set(CMAKE_CXX_STANDARD 17)           # 所有平台使用 C++17
set(CMAKE_CXX_STANDARD_REQUIRED ON) # 编译器不支持 C++17 则直接报错

if(MSVC)
    add_compile_options(/utf-8)      # 仅 MSVC 需要此选项
endif()
```

这里 `CMAKE_CXX_STANDARD 17` 在生成构建文件时会自动转换为对应编译器的选项：GCC/Clang → `-std=c++17`，MSVC → `/std:c++17`。作者不需要记住不同编译器的 flag 写法。`if(MSVC)` 条件编译只对 Windows 上的 MSVC 编译器添加 `/utf-8`，其他编译器不受影响。

第二：库依赖的声明式查找。

```
find_package(QT NAMES Qt6 Qt5 REQUIRED COMPONENTS Widgets Multimedia Core)
find_package(Qt${QT_VERSION_MAJOR} REQUIRED COMPONENTS Widgets Multimedia Core)
```

第一行查找 Qt 本身：优先找 Qt6，找不到就找 Qt5。如果两个都找不到，CMake 会在配置阶段直接报错并给出清晰的错误信息（“Could not find a package configuration file...”），而不是在链接阶段爆出几十行 `undefined reference` 错误。

第二行根据找到的版本导入对应模块。`Qt${QT_VERSION_MAJOR}` 在找到 Qt6 时展开为 `Qt6`，找到 Qt5 时展开为 `Qt5`。

第三：资源文件的集成。

```
qt_add_resources(GAME_RESOURCES Resources/game_character.qrc)
```

这行代码将 Qt 资源文件（.qrc）编译为 C++ 源代码并链接进可执行文件。所有图片和音效在运行时通过 `:/assets/...` 路径访问，不需要额外打包或配置资源搜索路径。

第四：目标与链接的声明式管理。

```
target_link_libraries(shooting_game PRIVATE
    Qt${QT_VERSION_MAJOR}::Widgets
    Qt${QT_VERSION_MAJOR}::Multimedia
    Qt${QT_VERSION_MAJOR}::Core
)
```

`target_link_libraries` 不是简单地拼凑 `-l` 选项——它会自动推导 `include` 路径（继承 Qt 模块的 `include` 目录）、链接库路径、以及必要的编译定义。`PRIVATE` 表示这三个模块是 `shooting_game` 内部使用的依赖，不会泄漏给其他链接了 `shooting_game` 的目标。

第五：平台特定打包配置。

```
set_target_properties(shooting_game PROPERTIES
    MACOSX_BUNDLE TRUE          # macOS: 打包为 .app bundle
    WIN32_EXECUTABLE TRUE       # Windows: 不弹出控制台窗口
)
```

同样的 CMake 语句在不同平台上产生不同的效果：macOS 上将可执行文件组织为 `shooting_game.app/Contents/MacOS/shooting_game` 的 bundle 结构，Windows 上将入口函数设为 `WinMain` 而非 `main` 以避免弹出控制台窗口（或对 MinGW 设置子系统为 Windows）。

第六：源码的自动化收集与 moc 预处理。

Qt 的信号槽需要元对象编译器（moc）——它会扫描头文件中包含 `Q_OBJECT` 宏的类，生成额外的 C++ 代码（信号的实现、类型信息、反射元数据）。手动运行 moc 并在编译时链接生成的代码极其繁琐。

```
set(CMAKE_AUTOMOC ON) # 自动运行 moc
set(CMAKE_AUTORCC ON) # 自动编译 .qrc 资源文件
set(CMAKE_AUTOUIC ON) # 自动编译 .ui 界面文件
```

这三行开启后，CMake 会自动扫描源码目录中含有 `Q_OBJECT` 的头文件并运行 moc，自动处理 .qrc，自动将 .ui 文件编译为 C++ 头文件。开发者只需写头文件，不需要手动管理这些中间生成物。

7.3 代码验证

验证项	输入/触发	预期行为	结论
平台可移植性	在 Windows (MinGW) 和 macOS 上分别配置	同一套 CMakeLists.txt，通过两个平台上分别找到对应 Qt 安装路径并生成正确的构建文件	通过
库缺失报错	卸载 Qt Multimedia 模块后重新 cmake	CMake 配置阶段报错 "Could not find Qt6Multimedia"，而非编译到链接阶段才报 undefined reference	通过
moc 自动处理	修改 playercontroller.h 中 signals 声明	重新编译时 moc 自动重新生成元对象代码，编译和链接无需手动干预	通过
头文件依赖追踪	修改 spellstate.h（被 player.h 包含，player.h 被 10+ 个 .cpp 包含）	只重新编译受影响的 .cpp，无需 clean rebuild	通过

7.4 替代方案分析

方案	优势	代价/为何未采用
手动 Makefile / 批处理脚本	无外部工具依赖，适合 2-3 个文件的极小项目	跨平台路径管理极其痛苦（两个平台两套脚本）；头文件依赖需要手动维护或借助 gcc -M；新增 .cpp 文件需要手动更新构建脚本
qmake（Qt 原生构建工具）	Qt 自带，默认配置简单	qmake 的功能集远少于 CMake；跨平台能力弱（实际是一个增强版 Makefile 生成器）；Qt 官方已明确 qmake 进入维护模式，推荐 CMake
Visual Studio 项目（.sln / .vcxproj）	Windows 上原生 IDE 支持最好；图形界面配置编译器选项	锁定 Windows + MSVC；需要另外维护 macOS 的 Makefile 或 Xcode 项目——两套构建配置。本项目不需要 Visual Studio 的 GUI 调试器（用 qDebug 即可）
当前方案（CMake）	平台无关的声明式构建描述；强大的 find_package 机制；Qt 官方推荐；社区生态最大	学习曲线稍陡（find_package 的语法和 target 属性的概念需要时间理解）。但对课程项目而言，CMakeLists.txt 全文仅 90 行，已经用最简功能集覆盖了需求

7.5 小结

CMake 在本项目中的角色是把“我要构建什么”和“怎么在这个平台上构建”分离。开发者只需要在 CMakeLists.txt 中声明源码列表、C++ 标准版本、依赖的 Qt 模块和目标平台属性，CMake 负责将这些声明转换为当前平台的原生构建指令。对于一个 40+ 文件、跨平台、依赖多模块 Qt 库的项目，这个抽象使得“换个平台编译”从一件需要重写构建脚本的大工程变成无需任何改动的一键操作。

A 编译与运行环境

A.1 开发与运行环境

- 操作系统：Windows 10/11（主力开发与测试环境）。macOS/Linux 上理论可编译（CMakeLists.txt 已配置跨平台），但未做完整测试。
- 编译器：MinGW GCC 11.2.0（Windows）。macOS 上使用 Apple Clang（Xcode Command Line Tools 自带），Linux 上使用 GCC/Clang。
- Qt 版本：Qt 6.5.3 LTS。兼容 Qt 6.2+ 任意版本。

- 构建系统: CMake 3.16+, Ninja (推荐) / Make
- C++ 标准: C++17

A.2 项目文件清单

提交包中与编译和运行相关的核心文件 (共 40+ 源文件):

CMakeLists.txt	# CMake 构建配置 (全文 90 行)
Resources/game_character.qrc	# Qt 资源文件 (图片、音效)
main.cpp	# 程序入口, 解析命令行参数
widget.h / widget.cpp	# 游戏主界面 + 60FPS 帧循环
player.h / player.cpp	# 角色实体 (属性、武器、词条、法术)
playercontroller.h / .cpp	# 输入意图转换与动作控制
entity.h / entity.cpp	# 实体基类 (飞物/掉落物共有接口)
ball.h / ball.cpp	# 实心球 (抛物线弹道)
bullet.h / bullet.cpp	# 子弹 (直线弹道, 步枪/狙击两种)
dropitem.h / dropitem.cpp	# 掉落物 (武器/词条卡/血瓶)
combatmanager.h / .cpp	# 战斗管理器 (碰撞→伤害→状态结算)
gameai.h / gameai.cpp	# AI 系统 (FSM + 人格 + 寻路)
map.h / map.cpp	# 地图碰撞体 + 平台间 BFS 寻路
animation.h / animation.cpp	# 精灵动画 (帧序列加载/播放)
soundmanager.h / .cpp	# 音效管理 (单例, 预加载)
gamesession.h	# 游戏会话配置结构体
spellstate.h	# 法术运行时状态
modifierdata.h	# 词条数据定义 (header-only)
modifieroverlay.h / .cpp	# 无尽模式词条三选一弹窗
mainwindow.h / mainwindow.cpp	# 主窗口 + 页面导航 (菜单/设置/游戏)
setupwidget.h / setupwidget.cpp	# 开局设置页 (模式/参数/法术选择)
endlessresultwidget.h / .cpp	# 无尽模式结算面板 + 排行榜
endlessrecordmanager.h / .cpp	# 无尽记录统计
CollisionResult.h	# 碰撞结果结构体

A.3 编译步骤

前置条件: 系统已安装 Qt 6.x (含 Widgets、Multimedia、Core 模块)、CMake 3.16+、支持 C++17 的编译器。

1. 打开终端, 进入项目根目录 (CMakeLists.txt 所在目录)。
2. 配置:

```
cmake -B build -G Ninja -DCMAKE_BUILD_TYPE=Release
```


若未安装 Ninja, 可省略 `-G Ninja`, CMake 使用平台默认生成器 (Windows 上 NMake 或 VS; macOS/Linux 上 Make)。

3. 编译:

```
cmake --build build --config Release
```

编译过程约需 1-2 分钟。若出现 `Could not find Qt6` 错误, 指定 Qt 安装路径:

```
cmake -B build -DCMAKE_PREFIX_PATH=/path/to/Qt/6.x/gcc_64
```

4. 运行:

```
# Windows
build\shooting_game.exe

# macOS
open build/shooting_game.app

# Linux
./build/shooting_game
```

5. 资源说明: 所有图片和音效通过 `.qrc` 编译进可执行文件, 无需外置资源目录。如运行时无音效, 检查 Qt Multimedia 模块是否正确安装 (Windows 上确保 `Qt6Multimedia.dll` 在 PATH 中)。

A.4 编译验证

在以下环境中成功编译并运行, 帧率稳定在约 60FPS:

- Windows 11 + MinGW GCC 11.2.0 + Qt 6.5.3 + Ninja

A.5 运行与操作说明

- **首次启动:** 点击“开始”进入设置页, 选择“人机对战”模式, 保持默认参数, 点击“开始对局”。
- **P1 键位:** W/A/S/D 移动, F 攻击, G/C 切换武器槽, Q 释放法术。
- **P2 键位 (双人模式):** I/J/K/L 移动, O 攻击, N 切换武器槽, U 释放法术。
- **结束对局:** 一方阵亡后显示结算画面, 按任意键返回主菜单。
- **演示视频:** 完整的玩法演示见提交附件中的视频文件。

结语

本项目从题目 3.19 “横版格斗对战游戏”出发，实现了一个功能完整的游戏系统。但这篇报告论述的核心不是“我们做了个游戏”，而是这个游戏为什么需要这六项技术。

六项技术之间存在内在关联：OOP 的继承和组合是模块化的基础（技术一），模块间的通信靠信号槽解耦（技术二），词条和法术通过数据驱动保持可扩展（技术三），AI 通过 FSM 实现结构化决策（技术四），所有模块在精心编排的时序中协同工作（技术五），最终由 CMake 屏蔽平台差异一键构建（技术六）。这六项技术不是并列的关系，而是一个从内向外的层次——从单个类的设计，到模块间通信，到系统的可扩展性和智能性，到运行时的执行编排，到最终的构建交付。

完成这个项目让我深刻理解了“游戏开发不仅是写代码，更是设计系统”的含义。每一行代码的背后，都有对可维护性、可扩展性和性能的权衡；每一个模块的设计，都需要考虑它与其他模块的交互和依赖。通过这次实践，我不仅巩固了 C++ 和 Qt 的基础知识，也学会了如何在有限的时间内做出合理的技术选择。在一个看似简单的横版格斗游戏背后，往往隐藏着一系列精心设计的技术方案，它们共同支撑了游戏的可玩性、可维护性和可扩展性。

本项目所有源码、资源文件和本文档的 LaTeX 源码均随提交一并提供，根据要求，编译完毕的可执行文件不被包含，需要助教/老师根据自己的环境自行编译。